

Automated Derivation of UML Sequence Diagrams from User Stories: Unleashing the Power of Generative AI vs. a Rule-Based Approach

Munima Jahan
munimajahan@yahoo.com
Department of Engineering,
Thompson Rivers University
Kamloops, BC, Canada

Mohammad Mahdi Hassan
mohammadhassan@upei.ca
University of Prince Edward Island
Charlottetown, Canada

Reza Golpayegani
reza.golpayegani@usask.ca
University of Saskatchewan
Saskatoon, Canada

Golshid Ranjbaran
ark048@usask.ca
University of Saskatchewan
Saskatoon, Canada

Chanchal Roy
chanchal.roy@usask.ca
University of Saskatchewan
Saskatoon, Canada

Banani Roy
University of Saskatchewan
Saskatoon, Canada

Kevin Schneider
University of Saskatchewan
Saskatoon, Canada

ABSTRACT

User stories are informal, non-technical descriptions of features from a user's perspective that guide collaboration and iterative development in Agile projects. However, ambiguities in user stories can lead to miscommunication among stakeholders. Design models, such as UML sequence diagrams, are essential for enhancing communication, clarifying system behavior, and improving the development process. This paper presents an automated approach for generating behavioral models specifically sequence diagrams from natural language requirements expressed as user stories. We also investigate the effectiveness of a Large Language Model (LLM) in using generative AI for this task. By applying our approach and ChatGPT to two benchmark datasets with the same set of user stories, we generated corresponding sequence diagrams for comparison. Expert evaluations in Software Engineering reveal that our approach effectively produces relevant, simplified diagrams for straightforward user stories, whereas the LLM tends to create more complex diagrams that sometimes go beyond the simplicity of the original user stories.

CCS CONCEPTS

• **Software and its engineering** → **Software design engineering**.

KEYWORDS

User Story, Sequence Diagram, Generative Model, Large Language Model, Model Generation, Natural Language Processing, Rule-based approach

ACM Reference Format:

Munima Jahan, Mohammad Mahdi Hassan, Reza Golpayegani, Golshid Ranjbaran, Chanchal Roy, Banani Roy, and Kevin Schneider. 2024. Automated Derivation of UML Sequence Diagrams from User Stories: Unleashing the Power of Generative AI vs. a Rule-Based Approach. In *ACM/IEEE 27th International Conference on Model Driven Engineering Languages and Systems (MODELS '24)*, September 22–27, 2024, Linz, Austria. ACM, New York, NY, USA, 11 pages. <https://doi.org/10.1145/3640310.3674081>

1 INTRODUCTION

Agile Software Development projects often use user stories to explore initial requirements. These stories, written from the end user's perspective in natural language, describe software features in an informal manner to convey the value to the customer and are growing in popularity [36]. They promote cross-team clarity on what to build, for whom, why, and when. Due to their simplicity, user stories are a common method for communicating product functionality. However, if defined incorrectly, they can lead to significant development challenges [17].

User stories (US) vary in format and style between organizations. However, literature recommends a variety of formats [44]. For example, one of the most popular templates used in industry [37, 44] is Cohn's template [4] which describes stories in the form "As a < actor/user > I want < goal/action > so that < reason/value >." (e.g., "As a customer I want to pay online by credit card so that I can keep track of my spending."). Agile practices also suggest including acceptance criteria in each story. Since this requires a more detailed analysis of stories, acceptance criteria are often not defined [11].

As user stories are written in an informal and more natural language, the requirements may be too vague for engineers to

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

MODELS '24, September 22–27, 2024, Linz, Austria

© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 979-8-4007-0504-5/24/09

<https://doi.org/10.1145/3640310.3674081>

work with specially for large and complex projects. Where as the visual models like Sequence Diagrams (SDs) can enhance the understanding of the behaviours of a system as well as automate the evaluation of the behaviours for any unexpected flaws. SDs are great tools at the start of a project as it shows the user what has to happen in a step-by-step fashion [12]. Although formal models are highly efficient for designing, testing, and verification of critical systems, generating models from Natural Language (NL) representation is complex, time-consuming and tedious. Consequently, both the scientific and industrial communities have shown an interest in automating the process of generating UML models from NL requirements. The UML 2.0 modelling language [18] such as UML SD has emerged as the standard for modelling software systems and attracted considerable attention of researchers and designers. This paper presents an approach to generate sequence diagrams from user stories. A sequence diagram shows how a group of objects collaborate to accomplish a task[23]. It represents the behavior of a single use case as a set of message transfer between system components.

Substantial efforts have been directed towards identifying suitable models from user stories that illustrate both the static and dynamic behaviors of a system. Various automated and semi-automated methods for generating UML models from user stories have been proposed by different researchers [10, 21, 28]. However, a review of the literature on automatic UML diagram generation indicates that only a few studies focus on generating SDs [10, 30]. UML sequence diagrams allegedly provide a simple enough syntax to express a wide variety of behaviors, including expected scenarios of good or bad behavior in systems under design [3]. They are considered the second most common UML diagram and have been informally utilized for several decades [1]. To acknowledge the importance of automation in behavioral modeling and to address some of the gaps in the literature, we propose an automated approach that constructs SDs from user stories (US) written in natural language. We use a rule-based approach and various NL processing techniques to identify the components of an SD and their relationships. A text script is then prepared, which can be run within the open-source tool PlantUML [14] to draw the actual diagram.

The paper presents notable contributions encapsulated as follows:

- Introduction of an automated approach for generating SDs from USs.
- Exploration of the efficacy of generative AI in modeling by generating SDs from user stories.
- Comparison of results obtained from a rule-based approach with those generated by LLMs like ChatGPT.

To evaluate our proposed tool, we utilized 100 user stories sourced from two benchmark datasets [5, 35]. Employing the LLM ChatGPT [47], we generated SDs for the same set of USs, subsequently comparing these results with our own generated SDs. This comparison aimed to harness the potential of LLM in SD generation. To assess the outcomes, we enlisted the expertise of two software professionals who evaluated them based on specific criteria centered on the foundational features of an SD.

According to the experts, our method demonstrates superiority by generating more accurate and simplified SDs for simple and

short user stories. In average our tool achieved 80% accuracy in object detection. However, ChatGPT exhibits a higher capability in recognizing context and adding more details to the diagram that may not be explicitly available within the US, making it more intricate. The rest of this paper is structured as follows: Section 2 describes the related work by briefly focusing on their scope and limitations. In Section 3, we provide preliminary definitions and set a stage for the methodology. The overall approach is presented in Section 4. The experimental results, as well as the findings and evaluation, are presented in Section 5. Finally, in Section 6, we present conclusions and future plans.

2 RELATED WORK

Several research have been carried on automated and semi-automated generation of UML diagrams from NL requirements [32, 40, 41]. Use Case Driven Development Assistant (UCDA) [40] tool helps in developing class diagrams, use-case models and also helps to visualize the models using Rational Rose tool. The tool uses syntactic analysis of the requirement statements to develop use-case diagrams. Linguistic assistance for Domain Analysis, LIDA [32] is a semi-automatic tool that helps designer in creating class diagrams. Several techniques have been developed to automatically generate SD from use cases [41, 48]. Takhur and Gupta transform textual use case descriptions written as numbered steps into UML sequence diagrams [41]. In all those works, the generated diagrams are limited in their expressive power and in many cases limited to strict input structure to be followed.

An automatic approach to derive SDs from use case scenarios (UCS) are presented in [19]. They represented a set of transformation rules for deriving SDs from NL UCSs. We have followed some of the techniques used in [19] for preprocessing and deriving the key elements of a SD from the textual descriptions using natural language processing (NLP). However, it is important to highlight the key distinction between the two approaches is paper [19] focuses on generating SDs from UCSs, whereas our paper generates SDs from USs. UCSs and USs serve as different tools for representing requirements artifacts. Given the fundamentally different nature and structure of these input texts, the algorithms for identifying components in each context differ significantly.

Works that generate UML models from USs include [10, 13, 21, 30]. An automatic transformation from USs to robustness diagram is proposed by Fabian et al. [11]. The approach aims at enabling requirement engineers and users to validate the main concepts and functional steps behind stories and discover malformed or redundant stories. A technique to automatically transform textual USs into visual UCS in the form of robustness diagrams is presented in [13]. This study utilises natural language processing and rule-based transformations. The authors of this paper [13] developed an algorithm that automatically creates multiple conceptual models when USs are fed to it. In order to do so, they supplement the standard structure of USs with acceptance criteria, as is common practice in behavior driven development (BDD) [38]. The conceptual models that are generated by their algorithm are: Domain Model [20], Process Model[46], Finite State Machine [18], and Use Case Model [18]. [29] presents an approach that automatically generate UML diagrams, namely class, use cases, and package diagrams. Prolog

rules are used to extract relationships between classes and eliminate those that are at risk of error. Lin et al. [25] and Wautelet et al. [45] build goal models from (annotated) USs, but these two techniques are mostly manual. Only a few research focus on generating SD from USs [10, 30]. The research study [10] presents an algorithm for automating the transformation of USs into SDs in the Scrum process, which can easily be used to generate test cases. This paper implements an algorithm that read a text file containing a set of USs and generates an XMI file for each USs. The resulting XMI file then transformed into a SD using UML2 tool SDK plugin for Eclipse. A problem with the proposed approach is that it is unable to detect actors with compound nouns and inclusion and extension relations between use cases.

Another approach for generating SDs from USs scenario is presented in [30]. The approach takes input as a set of USs in form of semi structured format, and construct US scenarios adopting from Behat cheat sheet format. The US scenario is processed by applying natural language processing to identify sequence diagram elements. The output of the system is a SD in the form of text code of Plant UML and portable network graphics (PNG) image format. To evaluate the result of study, the authors compare their output with manually generated SDs by the groups of system analyst to calculate the accuracy of object creation, message flow, and name of the object identification. However, this approach cannot extract entity objects or messages between the boundary and control objects.

The preceding literature review highlights a notable gap in research concerning the generation of Sequence Diagrams (SD) from USs, indicating a clear need for further advancement in this domain. Existing approaches predominantly focus on generating system-level SDs and often rely on constrained input formats. Moreover, the resultant diagrams exhibit limitations in their expressive capabilities, such as aggregating actions into one-way interactions and only partially identifying common behaviors. In our approach, we leverage various Natural Language Processing (NLP) techniques and heuristics to discern the fundamental components of high-level system SDs. Subsequently, we refine these diagrams to produce more detailed SDs.

Large Language Models (LLMs) have had a notable impact on various domains, with Software Engineering (SE) being no exception [9]. Recent publications have delved into the application of LLMs across diverse SE tasks. LLMs in Software Engineering (SE) offer an innovative approach, reframing various SE challenges as data, code, or text analysis tasks [43]. Their application has shown significant potential breakthroughs in tasks like code summarization [42], which generates concise natural language descriptions of code functionality, as well as in the generation of well-structured code and code artifacts, including annotations [24]. However, the literature review shows that only a few research focus on requirements engineering tasks [15]. To the best of our knowledge, there is no existing research exploring the generative capabilities of a Large Language Model (LLM) such as ChatGPT within the realm of SDs. We anticipate that this study will unveil new dimensions in the potential applications of LLM, particularly in requirements engineering, and in the generation of design models derived from requirements artifacts.

3 BACKGROUND AND RELATED TERMS

A US is a well-formed, short and simple description of a software requirement from the perspective of an end-user, written in an informal natural language. It is the main artifact used in the agile software development process to capture user requirements. USs may be expressed in more than 80 syntactic variants, from free-form texts to a (who/what/why) template [44]. However, 70% of practitioners use the template [26]: “As a *< type of user >*, I want *< goal >*, [so that *< some benefit >*]”, where the “so that” clause is optional. For example following the Cohn’s template [4], one may define a story like “As a user, I want to be able to create and edit a profile so that I am recognised by the website.” In our approach we define a US as a text T_i written in simple English sentences using the following format:

$$T_i = \text{As a } < U >, \text{ I want } < A >, \text{ so that } < B > .$$

where, U is the actor or user represented by a noun, A is the action represented as a set of words combining verb and modifiers, B is the benefit represented as a set of words.

A set of USs is defined as:

$$\tau = \{T_1, T_2, \dots, T_n\}$$

, where n is the total number of user stories in the set.

UML SDs [18] are interaction diagrams that detail how operations are carried out. They capture the interaction between actors and other system entities in the context of a collaboration. We define SD as a sequence of events represented by senders, receivers and messages collected from the USs. A sequence diagram Sd_i corresponding to the US T_i is defined as a set of tuples

$$Sd_i = \{(e_1, 1), (e_2, 2), \dots, (e_k, k)\}$$

where, e_k represents the k th event in the SD and k is the visual order of the event.

An event or an interaction e_k collected from the US T_i is a triple

$$e_k = (s_k, m_k, r_k)$$

where s_k , m_k , r_k are words collected from the sentence T_i and represents the sender, message and receiver respectively for the k th event in the corresponding SD Sd_i . The order of the events within the Sd_i is represented by k .

We denote the set of senders, receivers, and messages with S , R , and M respectively.

We also generalize S , R or M as system components O . We represent the set of components as,

$$O = \{o | o \in \{S|R|M\}\}$$

A set of SDs is defined as:

$$\lambda = \{Sd_1, Sd_2, \dots, Sd_n\}$$

, where n is the total number of SDs in the set.

Given a set of USs τ the problem addressed in this paper is to find the corresponding SDs λ as sequence of events. Each US T_i is represented by a single SD Sd_i .

4 PROPOSED METHOD

In this paper, we propose two distinct approaches for generating SDs from USs. The first approach outlines a sequential process, beginning with the preprocessing of the given set of textual USs and culminating in the generation of a script representing a SD, as depicted in Figure 1a. The second approach leverages an LLM, specifically ChatGPT, to generate SDs from USs, as illustrated in Figure 1b.

The input requirements are assumed to be expressed in a standard format of US and adhere to the following two constraints: (i) each sentence describes a single action or is written as a compound sentence using the conjunction 'and' to specify multiple tasks, and (ii) the stories are expressed correctly in terms of spelling and grammar.

As shown in Figure 1a, we follow a series of steps to generate SDs. Initially, the USs are collected as text and preprocessed using standard cleaning and preprocessing techniques. Subsequently, we apply natural language processing (NLP) tasks and heuristics to identify the elements required for an SD. The details of these steps are discussed in the following sections.

4.1 Preprocessing and Collecting noun phrases

Our approach starts with reading a text file containing a list of USs in the format "as a [Actor], i want to [Action], so that [Benefit]". The preprocessing phase mainly deals with compound sentences and grouping consecutive nouns. USs are often written as: "as a [Actor], i want to [Action], so that [Benefit1] and [Benefit2]". The story is then written as two different stories.

In natural language an object is often written as a noun phrase rather than a single noun. For example, "As a user, I want to be able to create and edit a personal profile so that I am recognised by the website". The words "personal profile" should be considered as a single entity. To ensure that an object is represented as a collection of nouns or a combination of a modifier and nouns, our approach scans all the consecutive nouns and constructs a single word such as `personal_profile`. It also checks any consecutive modifiers and nouns for combining words as a single entity.

To prevent any potential conflicts between upper and lower case words, the system first converts the content to all lower case. Next, it combines the words using the '_' symbol. We adhere to the procedure outlined in the literature [19], which allows for a maximum of three consecutive nouns or modifiers to be combined into a single word. This approach simplifies the process and helps avoid excessively long words. Additionally, we remove any symbols and punctuation marks as part of the standard cleaning process.

4.2 Identifying Sender, Receiver and Message

In order to extract the elements of a SD (sender, receiver and message) we define a set of specific NLP heuristics. We implement the proposed NLP heuristics using Stanford CoreNLP [33]. The reason for using CoreNLP is the excellent results it offers [39]. This phase comprises several sub-processing stages to process the natural language (English) statements including tokenization, Parts-of-Speech (POS) Tagging, Stemming and Lemmatization, Type Dependencies and Open Information Extraction. The extraction of the elements

are done in two steps. In the first step we collect high level system elements as actor, action and benefit using the following rules:

R1: Actor is the first noun phrase chunk that exists after the pattern "as a". It can be a noun, noun plural or compound noun (already grouped in the preprocessing phase). The words Actor and Benefit represent in a SD respectively as Sender and Message.

R2: If Action is represented as Action= Verb + [modifier] + Noun/obj then the verb represents the action/message and the noun represents Receiver.

R3: If Action is represented as Action= Verb then the verb represents the action/message and the Receiver is represented as System.

R4: The Benefit is represented as a separate event causally dependant on the previous action.

4.3 Refining the SD elements

After extracting the actor, action, and benefit from the USs, we further analyze these elements to identify additional components that can enhance the SD. The actor is designated as the sender of the first event in the SD. The action, which may be a single verb or a phrase, is examined to determine the correct sequence of interactions. If the receiver is not explicitly mentioned in the sentence, it is replaced with "System" by default. The process for identifying and refining the SD using these rules is detailed in Algorithm 1. This algorithm systematically applies the predefined rules to generate a coherent and accurate SD.

Algorithm 1: Find Events for a SD from a given text T

FindEvents (T, U, O)

input : A text $T = (w_1, w_2, \dots, w_n)$, the actor U of the text and a set of potential objects O

output : A sequence of events $Sd = (e_1, e_2, \dots, e_n)$

$s_i \leftarrow U$

foreach word $w_i \in T$ **do**

if $dep(w_i) = \text{root}$ **or** $dep(w_i) \in \text{obl}, \text{obj}$ **then**

$m_i \leftarrow m_i + w_i$;

 Find next w_k such that $t(w_k) \in \{NN, NNS\}$

and $dep(w_k) = \text{nmod}$;

$r_i \leftarrow w_k$;

end

if $r_i \notin O$ **then**

$r_i \leftarrow \text{System}$;

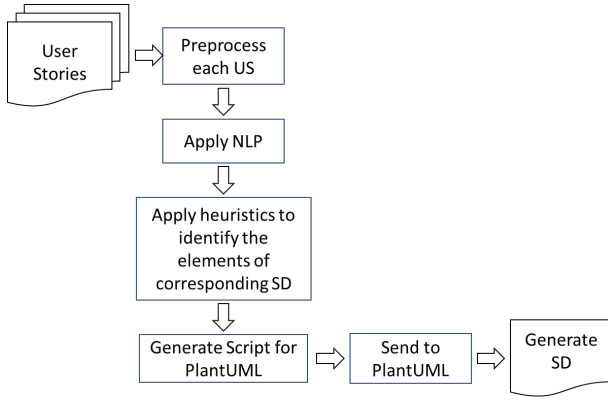
end

end

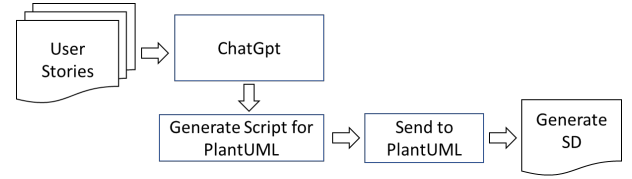
$e_i \leftarrow \text{createEvent}(s_i, m_i, r_i)$;

return e_i ;

The statement after the "so that" clause in a US often expresses another task that is causally dependant on the action described in the initial part of the US. We therefore analyze the benefit as a separate action and add as another event in the SD. The algorithm for processing the benefit within a single US to identify all components for a corresponding SD is provided in Algorithm 2.



(a) SD generating process in our approach.



(b) SD generating process from ChatGPT

Figure 1: Basic steps for generating SDs from USs.

Algorithm 2: Process a single user story S to find components for the SD

```

processUS ( $S, O$ )
  input : A user story as text  $S = (s_1, s_2, \dots, s_l)$ , A set of
    potential objects  $O$ 
  output : A sequence of events  $Sd = (e_1, e_2, \dots, e_n)$ 
   $U, A, B \leftarrow \text{getUserActionBenefit}(S)$ 
  foreach sentence  $s_i \in U$  do
     $e_i \leftarrow \text{FindEvents}(A, U, O)$ ;
     $\text{add}(e_i, Sd)$ ;
     $e_{i+1} \leftarrow \text{FindEvents}(B, U, O)$ ;
     $\text{add}(e_{i+1}, Sd)$ ;
  end
  return  $Sd$ ;

```

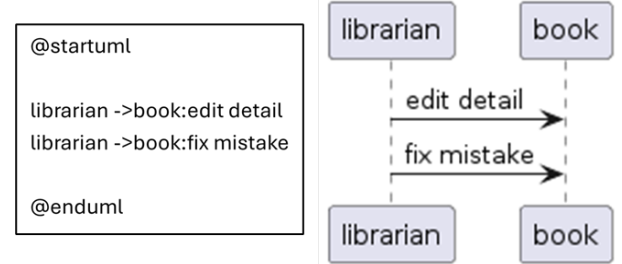


Figure 2: Sample PlantUML script and the corresponding SD generated from our system for the US "As a librarian, I want to edit the detail of books in the library so that I can fix in case of any mistake about them"

4.4 Illustrative Example

This section demonstrates the proposed approach with an example. We implemented our methodology as an executable program using Python 3.6 and the Stanford CoreNLP API. Our choice of Python as our primary development tool is motivated by its extensive array of libraries specifically crafted for machine learning (ML). Additionally, Python has solidified its position as the leading language for application development in the fields of data science and ML [2].

To illustrate our method let's consider a list of three USs for a library management system presented in Table 1.

The USs are read as a text input and preprocessed according to the steps described earlier. For example, the sentence "As a librarian, I want to keep track of book payments and subscriber profiles." is broken into two separate sentences "As a librarian, I want to keep track of book payments." and "As a librarian, I want to keep track of book subscriber profiles". The preprocessing also resolves any issues with coreferences. After the initial preprocessing is done the text is annotated by the Stanford NL parser and noun phrases are collected as mentioned in previous section. For example, the fourth sentence in the above example is transformed into "As a librarian, I want to keep track of book_payments and subscriber_profiles." The

elements of a SD (sender, receiver, messages) are then identified from each story according to the rules described in the previous section.

For the running example the components identified are shown in Table 2.

4.5 Generating Sequence Diagram

In this step, the information collected in Table 2 is used to generate a text script compatible with PlantUML [14] and saved as a text file. The generated text file is then fed to the PlantUML API to create the actual diagram as an image. The text script and the corresponding SD generated for the second US in the given example using PlantUML is presented in Figure 2.

The reasons for using PlantUML are twofold. First, PlantUML is a powerful tool that facilitates the creation of diagrams from text descriptions, making it quick and efficient to generate diagrams using simple text commands. Secondly, since all necessary information for generating a SD is collected in Table 2, different types of scripts can be generated based on this data. Additionally, generating actual images of the diagrams is beyond the scope of this research.

Table 1: Sample user stories for Library Management System.

SL	User Story
1	As a librarian, I want to edit suggestion list.
2	As a librarian, I want to edit the detail of books in the library so that I can fix in case of any mistake about them.
3	As a librarian, I want to edit the due date of loaned books so that I can change the date if the borrower informs in advance.

Table 2: Elements identified from the example stories.

SD	Sender	Message	Receiver
1	librarian	edit suggestion_list	System
2	librarian	edit detail	book
2	librarian	fix mistake	book
3	librarian	edit due_date	book
3	librarian	change date advance	System
4	librarian	keep track	book_payment

4.6 Generating SD using LLM (ChatGPT)

Large Language Models (LLMs) are driving significant transformations in the domain of Software Engineering (SE). The capability of these models to adeptly navigate complex tasks holds the potential to fundamentally reshape various SE practices and tools [15].

To examine the effectiveness of LLMs in generating SDs from textual requirements, we leverage ChatGPT's zero-shot learning capability. Specifically, we employ ChatGPT 3.5 in a zero-shot scenario to generate SDs represented in PlantUML script. ChatGPT has garnered widespread usage among researchers for zero-shot learning, exhibiting promising performance across diverse Natural Language Processing (NLP) tasks [34] as well as within the software engineering domain [16]. This motivated us to incorporate ChatGPT into our research endeavors. The workflow for generating SD from USs using ChatGPT is depicted in Figure 1b. Initially, we prompt ChatGPT with individual US and request it to produce the SD in PlantUML script format. Subsequently, we collect the generated response from ChatGPT. In the subsequent stage, we input the script into the PlantUML online tool to generate the actual diagrams.

The experimental findings suggest that ChatGPT is adept at extracting additional information pertinent to the context of user stories, thereby resulting in more comprehensive diagrams. However, in several cases, the inclusion of extra information leads to increased intricacy of the diagrams compared to the original user stories.

Nevertheless, our experiments clearly demonstrate that LLMs, such as ChatGPT, have considerable potential for implementation in a model-driven design approach. They can serve as valuable tools for automating various tedious requirements engineering tasks, including the generation of design models from natural language documents. The SD generated by ChatGPT for the US "As a Data Consuming User, I want to be able to search any dataset published and publicly accessible by their title and metadata, So that I can find the datasets I'm interested in." is illustrated in Figure 3.

As ChatGPT cannot directly construct the diagram, we requested it to generate the PlantUML script for the SD corresponding to the given US. Subsequently, we generated the actual diagram from the

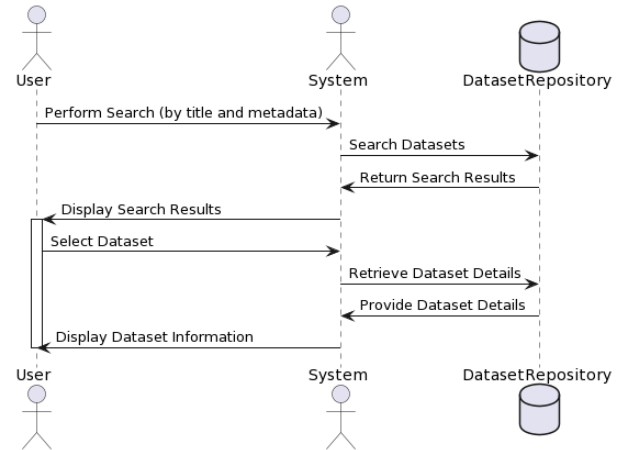


Figure 3: ChatGPT generated SD for the US "As a Data Consuming User, I want to be able to search any dataset published and publicly accessible by their title and metadata, So that I can find the datasets I'm interested in."

script using the PlantUML tool [14]. Figure 3 distinctly illustrates the efficiency with which ChatGPT extracts contextual information.

5 EXPERIMENTAL RESULTS AND EVALUATION

In this section, we explore the experimental results and the evaluation process of our approach. Additionally, we provide insights into the data collection and preparation procedures. Our proposed methodology is implemented as an executable program utilizing Python 3.6 and the Stanford CoreNLP API. To assess the effectiveness of our method, we employed three distinct approaches, outlined below:

- Manual evaluation by expert opinion
- Comparing with existing literature
- Comparing with LLM (ChatGPT) generated output.

Subsequent sections will delve into each evaluation type in detail.

5.1 Manual Evaluation

To assess the efficacy of our approach, we utilized two publicly available benchmark datasets [5, 35]. The first dataset [5] comprises 22 sets of requirements each containing 50+ user stories. These requirements were sourced from online platforms or obtained from software companies with explicit permission for disclosure. The second dataset [35] consists of user stories pertaining to two distinct systems, namely "Library Management" and "Restaurant Management," encompassing more than 350 user stories for each system. We utilize these datasets because they are publicly available and have been employed by various researchers in previous studies [6, 7, 22]. To evaluate the performance of our tool compared to ChatGPT, we randomly selected 100 user stories from the two datasets. These user stories were then inputted into both our system and ChatGPT, resulting in the generation of corresponding SDs. When determining the sample size of user stories for our evaluation, we took into account that assessing the entire dataset manually would be impractical. Therefore, we opted for a sample size of 100, which we deemed sufficient based on previous research studies [8]. As highlighted in the related work section, there is a lack of fully automated approaches for generating SDs from textual user stories, similar to our method. With no baseline dataset or reported results available for comparison, our focus is on methodologically evaluating the behavioral characteristics of the generated SDs. Hence, we engaged two domain experts, graduate students in the Department of Computer Science with several years of experience in the software industry, to evaluate the results produced by our system.

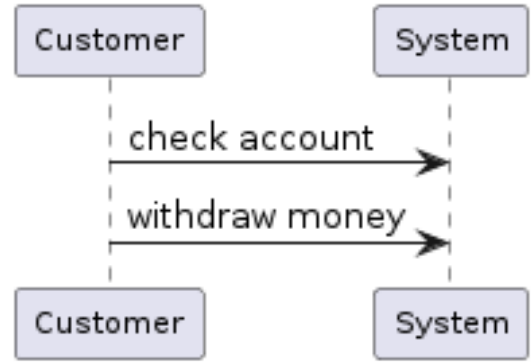
5.1.1 Evaluation process. The two domain experts were employed to evaluate the quality of the SDs based on the fundamental elements of a SD such as objects, messages, and interactions. They were tasked with rating each diagram on its relevance to the provided user stories by answering the five questions presented in Table 3. Our evaluation focuses on three key criteria:

- Number of relevant and irrelevant objects identified
- Number of relevant and irrelevant messages identified
- Number relevant and irrelevant interactions identified

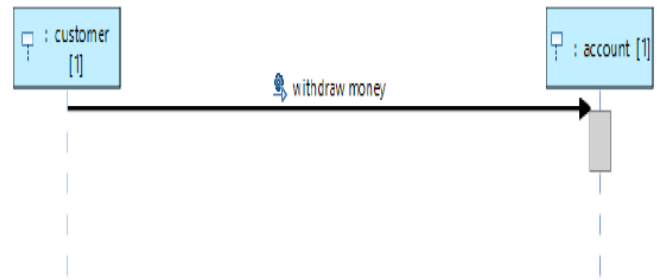
Based on the relevant and irrelevant components, experts are asked to rate the SDs by answering the given questions. Responses to the first and fifth questions (see Table 3) were limited to binary choices ('yes' or 'no'). The remaining questions were rated on a scale from 0 to 10, with 0 indicating no correct element identification and 10 indicating 100% correctness.

5.1.2 Evaluation results. The evaluation results from the experts are presented in Table 3.

From Table 3, we observe that out of the 100 SDs examined, 87 were identified as relevant (row: 1, column:3) for our rule-based approach. Furthermore, experts agreed that our tool could save time for software engineers in 72 cases. Notably, the average rating for accurately identifying objects was 8.89 out of 10. The ratings for correctly identified messages and interactions were above 7, indicating an acceptable level of accuracy. These results highlight the tool's effectiveness in identifying key elements of SDs, making it a valuable asset for improving the efficiency and accuracy of software design processes.



(a) SD generated from our tool.



(b) SD generated generated by [10]

Figure 4: SD generated for the user story: "As a customer, i want to check my account, so that withdraw money"

5.2 Comparing with existing literature

Manual evaluation can be subjective, depending on the expertise of the evaluators. To enhance the credibility of our system, we compared its results with those from existing literature. However, as detailed in the related work section, only a few fully automated approaches exist for generating SDs from textual user stories similar to ours, and these tools are not publicly available. Therefore, we compared our method with the results from [10], which presents an algorithm for automating the transformation of user stories into SDs in the Scrum process. For the comparison, we utilized the two example stories provided in this article and compared the SDs generated by our method with those they produced. Part of the result is depicted in Figure 4.

It is obvious from the Figure 4 that our approach produces more detailed and accurate SDs (SD) compared to the method introduced in [10].

5.3 Comparing with LLM (ChatGPT) generated results

To evaluate our approach and investigate the generative capabilities of LLMs in generating SD from textual user stories, we provided the same set of user stories used in our system to ChatGPT. We used the following prompt for the ChatGPT:

Table 3: Evaluation results based on expert opinion

Q. ID	Question	Our Tool	GPT
QE1	Is the generated Sequence Diagram (SD) relevant to the given User Story (US)?	87	100
QE2	Rate the accuracy of object representation and interactions between them on a scale of 1 to 10.	8.89	9.56
QE3	Rate the accuracy of message and interaction representation on a scale of 1 to 10.	7.34	9.18
QE4	Rate the accuracy of the message sequence order on a scale of 1 to 10.	7.13	9.17
QE5	Do you think this tool will save time for the Software Engineers to model SD for this particular user story?	72	84

"This is a user story: <insert user story here>. Generate a detailed sequence diagram in Plant UML format."

We provided all 100 user stories to the large language model (LLM) as zero shot prompt. Initially we experimented with few-shot prompts using 15 user stories and observed no differences in the generated SDs across multiple attempts. During the actual experimentation phase, we refrained from making multiple attempts because GPT-3.5 consistently generated SDs in Plant UML format without syntax errors. Additionally, repeated attempts did not yield any variations in the SDs. Furthermore, we used a zero-shot approach to maintain consistency with the rule-based method. In the rule-based approach, each US is processed individually to extract components for the corresponding SD. Therefore, we applied the same zero-shot principle to ChatGPT to ensure uniformity.

The resulting SDs from ChatGPT were assessed by the same two domain experts using the identical set of questions outlined in the previous section. The histograms displaying the ratings for both approaches are depicted in Figure 5.

The Figure illustrates that the SDs generated by the LLM generally received higher ratings compared to those produced by our tool in most cases. ChatGPT gathers and incorporates additional context not explicitly stated in the user stories, resulting in more sophisticated SDs. However, a closer examination of SDs generated by both systems reveals that our tool tends to produce simpler and more straightforward SDs for short and uncomplicated user stories. In contrast, ChatGPT often includes more context-related elements, leading to overly complex SDs with extraneous information.

For instance, the SDs generated for the US "As an employee, I want to see all store orders" by both our tool and ChatGPT, shown in Figure 6, demonstrate ChatGPT's tendency to overcomplicate the SDs, whereas our tool provides a more straightforward representation. The histogram further shows that these short user stories received lower scores from experts in most evaluation criteria and displays a pattern in the histogram. However, This pattern arises from the lower scores given by experts for the same short user stories across all evaluation criteria and does not convey any meaningful insight into the overall performance of the LLM.

We also want to note here that we compared actual diagrams rather than the script generated from both tools since we evaluated our results with domain experts, who found that visual diagrams are easier to compare than scripts.

5.4 Threat to the Validity

The limitations of our proposal are discussed in this section in terms of internal threats, construct threats and external threats.

Internal Validity: With regard to analyzing the sentence structures the rules used in our approach may not be exhausted to include

all possible types. However, we consider that the user stories are written in a standard format and we have used different heuristics to ensure that any sentence written in the predefined format is covered by our method.

Construct Validity: When evaluating the quality of the sequence diagrams, it's important to consider that it may be subject to the knowledge and expertise of the domain experts. To mitigate bias and limitations, we employed two experts who initially evaluated the results independently. Subsequently, they compared their assessments, resolving any discrepancies through discussion. The combined result of their evaluations is deemed to be more robust and reliable.

External Validity: As parts of our approach, we use the Stanford parser to identify SD components using POS tags and TDs, which can be considered a threat to its validity. The parser generates POS tags with accuracy about 97% [41] and type dependency with accuracy 84.2% [27]. Even a minor grammatical error can affect the performance of our algorithm by incorrectly identifying the POS tag. However, we assume that the user stories are simple sentences and easy to write while following the standard templates. Additionally, there are sophisticated text editors and tools [31] that can check for spelling, grammar, punctuation, clarity, engagement, and delivery errors to minimize writing errors.

Relying solely on ChatGPT can indeed be considered a limitation, as the results may vary across different Generative AI techniques. While ChatGPT is the most popular Large Language Model (LLM), exploring other LLMs in the future can help mitigate this limitation and provide a more comprehensive understanding of the capabilities and limitations of various generative AI approaches.

6 CONCLUSION AND FUTURE WORK

User stories are effective means for expressing user requirements from the view point of a user and gaining popularity among the developers. However, user stories often rely on informal language and can be open to interpretation resulting in a miscommunication between the development team and stakeholders. In contrast, design models like UML SDs enhance communication, clarify system behavior, and improve software quality. SDs visually represent interactions and message flows, aiding understanding among stakeholders. However, generating visual models from specifications written in natural language is tedious and time consuming. Automating the process of generating behavioural model like SD from requirements such as US is a complex and hardly addressed in the literature. In this paper we presented an approach to generate SD from US written in English. The proposed approach is proven to be fully automated and is able to generate more detailed SD

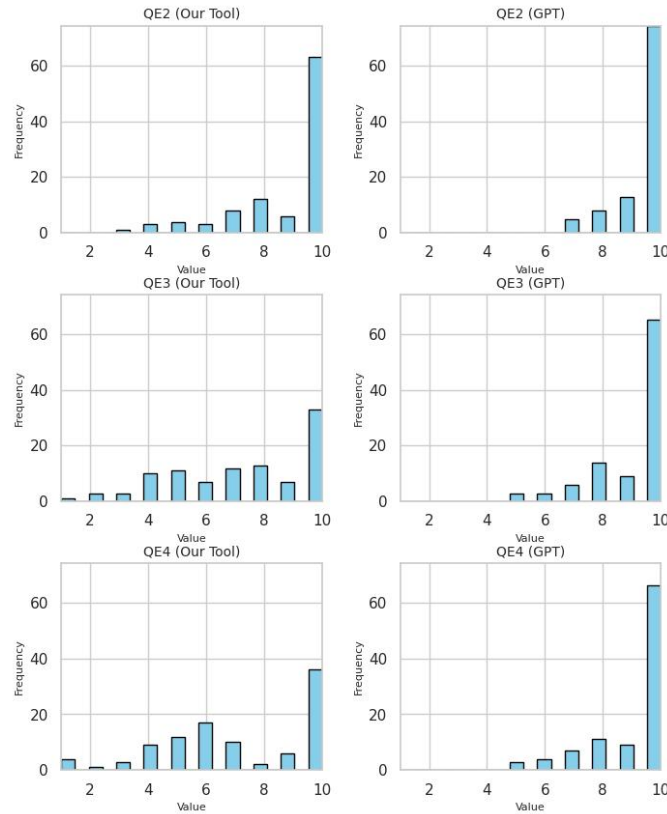


Figure 5: Comparison between the ratings for generated SDs from our approach to that of ChatGPT.

compared to the existing works [10, 30]. Based on expert evaluations, our system demonstrated an accuracy rate of approximately 70-80% in generating objects, messages, and interactions. Notably, its performance diminishes when handling user stories with complex sentence structures—an area we identify for enhancement and inclusion in our future plans.

This study also explores generative capability of an LLM, particularly ChatGPT, in the context of generating SDs from user stories and compares its output with those generated by our rule-based approach. The diagrams' quality is assessed by two domain experts. Results indicate that ChatGPT-generated SDs are highly relevant and accurate. While comparing our results with the result obtained from ChatGPT we observed that ChatGPT can provide more information to generate a detailed SD compared to our system. However, in some cases our method works better when the US is simple and short providing more relevant results than ChatGPT. These findings suggest ChatGPT's promising potential to support software engineering, particularly in model-driven development. Further exploration of ChatGPT's capabilities in various software engineering activities is recommended to better utilize the advanced features of LLMs and encourage their broader adoption in natural language-based software engineering practices.

Moving forward, our focus for future work involves developing a tool that incorporates the strengths of LLM and the rule-based approach to achieve better results. Additionally, we aim to create a

user-friendly application and make our system publicly available, aligning with our broader future plans. The source code, dataset used for this research, along with the evaluation data, can be accessed at: https://github.com/Jahan2021/From_User_Stories_to_SD.

ACKNOWLEDGMENTS

This research is partially supported by NSERC Discovery, CFI-JELF, and NSERC CREATE graduate program on Software Analytics Research (SOAR) grants and Living Skies Postdoctoral Fellows Program of University of Saskatchewan.

REFERENCES

- [1] Sabah Al-Fedaghi. 2021. UML sequence diagram: an alternative model. *arXiv preprint arXiv:2105.15152* (2021).
- [2] Oscar Castro, Pierrick Bruneau, Jean-Sébastien Sottet, and Dario Torregrossa. 2023. Landscape of High-Performance Python to Develop Data Science and Machine Learning Applications. *Comput. Surveys* 56, 3 (2023), 1–30.
- [3] Xiaohong Chen, Frédéric Mallet, and Xiaoshan Liu. 2020. Formally Verifying Sequence Diagrams for Safety Critical Systems. In *2020 International Symposium on Theoretical Aspects of Software Engineering (TASE)*. 217–224. <https://doi.org/10.1109/TASE49443.2020.00037>
- [4] Mike Cohn. 2004. *User stories applied: For agile software development*. Addison-Wesley Professional.
- [5] Fabiano Dalpiaz. 2018. Requirements data sets (user stories). *Mendeley Data*, v1 (2018).
- [6] Fabiano Dalpiaz, Patrizia Gieske, and Arnon Sturm. 2021. On deriving conceptual models from user requirements: An empirical study. *Information and Software Technology* 131 (03 2021), 106484. <https://doi.org/10.1016/j.infsof.2020.106484>

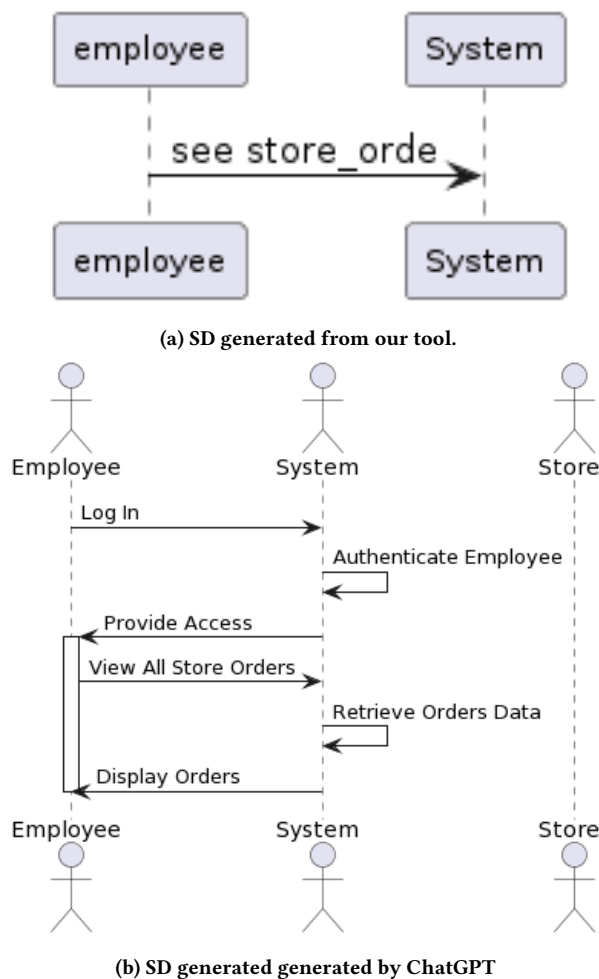


Figure 6: SD generated for the user story: "As an employee, I want to see all store orders."

- [7] Fabiano Dalpiaz, Ivor Schalk, Sjaak Brinkkemper, Fatma Aydemir, and Garm Lucassen. 2018. Detecting Terminological Ambiguity in User Stories: Tool and Experimentation. *Information and Software Technology* 110 (12 2018). <https://doi.org/10.1016/j.infsof.2018.12.007>
- [8] Joost de Winter, Dimitra Dodou, and Peter Wieringa. 2009. Exploratory Factor Analysis With Small Sample Sizes. *Multivariate Behavioral Research* 44 (04 2009), 147–181. <https://doi.org/10.1080/00273170902794206>
- [9] ROUABHIA Djaber and HADJADJ Ismail. 2023. AI as a Co-Engineer: A Case Study of ChatGPT in Software Lifecycle. (2023).
- [10] Meryem Elallaoui, Khalid Nafil, and Raja Touahni. 2015. Automatic generation of UML sequence diagrams from user stories in Scrum process. In *2015 10th international conference on intelligent systems: theories and applications (SITA)*. IEEE, 1–6.
- [11] Fabian Gilson, Matthias Galster, and François Georis. 2020. Generating use case scenarios from user stories. In *Proceedings of the international conference on software and system processes*. 31–40.
- [12] Gerhard Greeff and Ranjan Ghoshal. 2004. *Practical E-manufacturing and supply chain management*. Elsevier.
- [13] Abhimanyu Gupta, Geert Poels, and Palash Bera. 2019. Creation of Multiple Conceptual Models from User Stories—A Natural Language Processing Approach. In *Advances in Conceptual Modeling: ER 2019 Workshops FAIR, MREBA, EmpER, MoBiD, OntoCom, and ER Doctoral Symposium Papers, Salvador, Brazil, November 4–7, 2019, Proceedings 38*. Springer, 47–57.
- [14] Kwai Hing Heung. 2013. A tool for generating UML diagram from source code. (2013). <https://plantuml.com/sequence-diagram>
- [15] Xinyi Hou, Yanjie Zhao, Yue Liu, Zhou Yang, Kailong Wang, Li Li, Xiapu Luo, David Lo, John Grundy, and Haoyu Wang. 2023. Large language models for software engineering: A systematic literature review. *arXiv preprint arXiv:2308.10620* (2023).
- [16] Xinying Hou, Yanjie Zhao, Yue Liu, Zhou Yang, Kailong Wang, Li Li, Xiapu Luo, David Lo, John C. Grundy, and Haoyu Wang. 2023. Large Language Models for Software Engineering: A Systematic Literature Review. *ArXiv abs/2308.10620* (2023). <https://api.semanticscholar.org/CorpusID:261048648>
- [17] Irum Inayat, Siti Salwah Salim, Sabrina Marczak, Maya Daneva, and Shahabodin Shamshirband. 2015. A systematic literature review on agile requirements engineering practices and challenges. *Computers in human behavior* 51 (2015), 915–929.
- [18] Ivar Jacobson and James Rumbaugh Grady Booch. 2021. The unified modeling language reference manual. (2021).
- [19] Munima Jahan, Zahra Shakeri Hossein Abad, and Behrouz Far. 2021. Generating sequence diagram from natural language requirements. In *2021 IEEE 29th International Requirements Engineering Conference Workshops (REW)*. IEEE, 39–48.
- [20] Ian M Kinchin, David B Hay, and Alan Adams. 2000. How a qualitative approach to concept map analysis can be used to aid learning by illustrating patterns of conceptual development. *Educational research* 42, 1 (2000), 43–57.
- [21] Takwa Kochbati, Shuai Li, Sébastien Gérard, and Chokri Mraidha. 2021. From User Stories to Models: A Machine Learning Empowered Automation. *MODELSWARD* 10 (2021), 0010197800280040.
- [22] Salih Köse and Fatma Aydemir. 2023. A Framework To Improve User Story Sets Through Collaboration. (01 2023).
- [23] Liwu Li. 2000. Translating use cases to sequence diagrams. In *Proceedings ASE 2000. Fifteenth IEEE International Conference on Automated Software Engineering*. IEEE, 293–296.
- [24] Yuding Liang and Kenny Zhu. 2018. Automatic generation of text descriptive comments for code blocks. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 32.
- [25] Jun Lin, Han Yu, Zhiqi Shen, and Chunyan Miao. 2014. Using goal net to model user stories in agile software development. In *15th IEEE/ACIS International Conference on Software Engineering, Artificial Intelligence, Networking and Parallel/Distributed Computing (SNPD)*. IEEE, 1–6.
- [26] Garm Lucassen, Fabiano Dalpiaz, Jan Martijn EM van der Werf, and Sjaak Brinkkemper. 2016. The use and effectiveness of user stories in practice. In *Requirements Engineering: Foundation for Software Quality: 22nd International Working Conference, REFSQ 2016, Gothenburg, Sweden, March 14–17, 2016, Proceedings 22*. Springer, 205–222.
- [27] Christopher D Manning, Mihai Surdeanu, John Bauer, Jenny Rose Finkel, Steven Bethard, and David McClosky. 2014. The Stanford CoreNLP natural language processing toolkit. In *Proceedings of 52nd annual meeting of the association for computational linguistics: system demonstrations*. 55–60.
- [28] Samia Nasiri, Yassine Rhazali, and Mohammed Lahmer. 2021. Towards a generation of class diagram from user stories in agile methods. In *Advancements in Model-Driven Architecture in Software Engineering*. IGI Global, 135–159.
- [29] Samia Nasiri, Yassine Rhazali, Mohammed Lahmer, and Amina Adadi. 2021. From user stories to UML diagrams driven by ontological and production model. *International Journal of Advanced Computer Science and Applications* 12, 6 (2021).
- [30] Ahmad Ikil Nur, Faris Fasyadin, Imelda Adinda Febriansyah, Muhammad Alvian Yulianto, Ibaneza Griffint Secoria, Amelia Nagatha, Wilona Fadyah Trinaldy, Badrus Zaman, and Indra Kharisma Raharjana. 2023. User scenario 2 seq: Automated system to generate sequence diagram from user stories scenario. In *AIP Conference Proceedings*, Vol. 2554. AIP Publishing LLC, 040007.
- [31] Ruth O'Neill and Alex Russell. 2019. Stop! Grammar time: University students' perceptions of the automated feedback program Grammarly. *Australasian Journal of Educational Technology* 35, 1 (2019).
- [32] Scott P Overmyer, L Benoit, and R Owen. 2001. Conceptual modeling through linguistic analysis using LIDA. In *Proceedings of the 23rd International Conference on Software Engineering, ICSE 2001*. IEEE, 401–410.
- [33] Peng Qi, Yuhao Zhang, Yuhui Zhang, Jason Bolton, and Christopher D Manning. 2020. Stanza: A Python natural language processing toolkit for many human languages. *arXiv preprint arXiv:2003.07082* (2020).
- [34] Kun Ruan, Xiaohong Chen, and Zhi Jin. 2023. Requirements Modeling Aided by ChatGPT: An Experience in Embedded Systems. In *2023 IEEE 31st International Requirements Engineering Conference Workshops (REW)*. IEEE, 170–177.
- [35] Fatma Başak Aydemir Salih Göktaş Köse. 2023. A User Story Dataset for Library and Restaurant Management Systems. *Zenodo* (2023).
- [36] Eva-Maria Schön, Jörg Thomaschewski, and María José Escalona. 2017. Agile Requirements Engineering: A systematic literature review. *Computer standards & interfaces* 49 (2017), 79–91.
- [37] Todd Sedano, Paul Ralph, and Cécile Péraire. 2019. The product backlog. In *2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE)*. IEEE, 200–211.
- [38] JF Smart. 2014. BDD in Action-Behavior-Driven Development for the whole software lifecycle, 2014. *Greenwich: Manning Publications* (2014).

- [39] Min Song and Tamy Chambers. 2014. Text mining with the Stanford CoreNLP. In *Measuring scholarly impact: Methods and practice*. Springer, 215–234.
- [40] Kalaivani Subramaniam, Dong Liu, Behrouz Homayoun Far, and Armin Eberlein. 2004. UCDA: Use Case Driven Development Assistant Tool for Class Model Generation.. In *SEKE*, Vol. 7804. Citeseer, 16th.
- [41] Jitendra Singh Thakur and Atul Gupta. 2014. Automatic generation of sequence diagram from use case specification. In *Proceedings of the 7th India Software Engineering Conference*. 1–6.
- [42] Yao Wan, Zhou Zhao, Min Yang, Guandong Xu, Haochao Ying, Jian Wu, and Philip S Yu. 2018. Improving automatic source code summarization via deep reinforcement learning. In *Proceedings of the 33rd ACM/IEEE international conference on automated software engineering*. 397–407.
- [43] Simin Wang, Liguang Huang, Amiao Gao, Jidong Ge, Tengfei Zhang, Haitao Feng, Ishna Satyarth, Ming Li, He Zhang, and Vincent Ng. 2022. Machine/deep learning for software engineering: A systematic literature review. *IEEE Transactions on Software Engineering* 49, 3 (2022), 1188–1231.
- [44] Yves Wautelet, Samedi Heng, Manuel Kolp, and Isabelle Mirbel. 2014. Unifying and extending user story models. In *Advanced Information Systems Engineering: 26th International Conference, CAiSE 2014, Thessaloniki, Greece, June 16–20, 2014. Proceedings 26*. Springer, 211–225.
- [45] Yves Wautelet, Samedi Heng, Manuel Kolp, Isabelle Mirbel, and Stephan Poelmans. 2016. Building a rationale diagram for evaluating user story sets. In *2016 IEEE Tenth International Conference on Research Challenges in Information Science (RCIS)*. IEEE, 1–12.
- [46] Stephen A White et al. 2004. Business process modeling notation (bpmn).
- [47] Tianyu Wu, Shizhu He, Jingping Liu, Siqi Sun, Kang Liu, Qing-Long Han, and Yang Tang. 2023. A brief overview of ChatGPT: The history, status quo and potential future development. *IEEE/CAA Journal of Automatica Sinica* 10, 5 (2023), 1122–1136.
- [48] Tao Yue, Lionel C Briand, and Yvan Labiche. 2010. An automated approach to transform use cases into activity diagrams. In *European Conference on Modelling Foundations and Applications*. Springer, 337–353.